lstparseR: Tidy Parsing of NONMEM Listing Files

Raban Heller* Simeon R"{u}desheim† Dominik Selzer‡

2026-03-31

Contents

1 Introduction **2**

2 Installation **2**

3 Quick Start **2**

4 Reading Listing Files **2**

5 Extracting Parameters **3**

5.1 THETA — Fixed Effects 3

5.2 OMEGA — Random Effects 4

5.3 SIGMA — Residual Error 4

5.4 Objective Function Value 4

5.5 Condition Number 5

5.6 All Parameters at Once 5

6 Handling Runs Without a Covariance Step **5**

7 Batch Processing **6**

8 Function Reference **7**

9 Supported Estimation Methods **7**

10 Interactive Shiny Application **7**

11 Session Info **7**

*Charit'{e} – Universit"{a}tsmedizin Berlin, raban.heller@charite.de
†Saarland University, simeon.ruedesheim@uni-saarland.de
‡Saarland University, dominik.selzer@uni-saarland.de

1 Introduction

Population pharmacokinetic and pharmacodynamic (PK/PD) modelling with NONMEM produces listing files (`.lst`) that contain parameter estimates, standard errors, objective function values, eigenvalues, and shrinkage statistics in a structured but complex text format. Extracting these quantities for downstream analysis — model comparison tables, forest plots, covariate analyses — typically requires manual copy-paste or ad-hoc scripts that are fragile and error-prone.

lstparseR provides a clean, validated interface for reading `.lst` files and returning tidy data frames. It is designed with two goals:

1. **Robustness.** All parsers return `NA` for unavailable quantities rather than raising errors. This makes the package safe for batch workflows where some NONMEM runs may have failed or been terminated early.
2. **Tidiness.** All output is returned as tibbles with consistent, lowercase column names (`parameter`, `estimate`, `se`, `rse`, `shrinkage`) ready for binding across runs with `dplyr` or `purrr`.

The package supports all major NONMEM estimation methods: FOCE-I, FOCE, FO, SAEM, IMP, IMPMAP, and Bayesian analysis.

2 Installation

Install the development version from GitHub:

```
# install.packages("remotes")
remotes::install_github("Clinical-Pharmacy-Saarland-University/lstparseR")
```

3 Quick Start

```
library(lstparseR)

path <- system.file("testdata", "full_cov.lst", package = "lstparseR")
lst  <- read_lst_file(path)
lst
#> <lst> NONMEM listing file: 1321 lines
```

```
result <- fetch_all(lst)
names(result)
#> [1] "thetas" "etas"   "sigmas" "ofv"    "condn"
```

4 Reading Listing Files

`read_lst_file()` reads a `.lst` file into an S3 object of class `"lst"`. This is simply a character vector (one element per line) with the class attribute attached. All `fetch_*` functions require this class as input.

```
lst <- read_lst_file(
  system.file("testdata", "full_cov.lst", package = "lstparseR")
)
summary(lst)
#> <lst> NONMEM listing file
#>   Lines            : 1321
#>   Estimation method: FIRST ORDER CONDITIONAL ESTIMATION WITH INTERACTION
#>   Covariance step  : TRUE
```

5 Extracting Parameters

5.1 THETA — Fixed Effects

`fetch_thetas()` extracts the THETA parameter estimates from the *FINAL PARAMETER ESTIMATE* section. When the covariance step was completed, standard errors (SE) and relative standard errors (RSE, in percent) are also reported.

```
fetch_thetas(lst)
#> # A tibble: 12 x 4
#>    parameter     estimate       se     rse
#>    <chr>            <dbl>    <dbl>   <dbl>
#>  1 TH_1          34.1     3.37e+ 0 9.88e 0
#>  2 TH_2      387000       5.41e+ 7 1.40e 4
#>  3 TH_3           8.05    5.2 e- 1 6.46e 0
#>  4 TH_4           1.78    1.49e- 1 8.37e 0
#>  5 TH_5           0.498   2.78e- 2 5.58e 0
#>  6 TH_6           0.362   4.81e- 1 1.33e 2
#>  7 TH_7          -0.171   1.35e- 1 7.89e 1
#>  8 TH_8          -0.0314  2.69e- 1 8.57e 2
#>  9 TH_9          -0.163   2.78e- 1 1.71e 2
#> 10 TH10          -0.23    1.88e- 1 8.17e 1
#> 11 TH11           0.00539 1.98e- 2 3.67e 2
#> 12 TH12           0.2     1.90e+74 9.50e76
```

The `digits` argument rounds RSE values:

```
fetch_thetas(lst, digits = 1)
#> # A tibble: 12 x 4
#>    parameter     estimate       se     rse
#>    <chr>            <dbl>    <dbl>   <dbl>
#>  1 TH_1          34.1     3.37e+ 0 9.9 e 0
#>  2 TH_2      387000       5.41e+ 7 1.40e 4
#>  3 TH_3           8.05    5.2 e- 1 6.5 e 0
#>  4 TH_4           1.78    1.49e- 1 8.4 e 0
#>  5 TH_5           0.498   2.78e- 2 5.6 e 0
#>  6 TH_6           0.362   4.81e- 1 1.33e 2
#>  7 TH_7          -0.171   1.35e- 1 7.89e 1
#>  8 TH_8          -0.0314  2.69e- 1 8.57e 2
#>  9 TH_9          -0.163   2.78e- 1 1.71e 2
#> 10 TH10          -0.23    1.88e- 1 8.17e 1
```

```
#> 11 TH11           0.00539 1.98e- 2 3.67e 2
#> 12 TH12           0.2     1.90e+74 9.50e76
```

5.2 OMEGA — Random Effects

`fetch_etas()` extracts the diagonal of the OMEGA covariance matrix (variance of inter-individual random effects). ETA shrinkage is included when reported by NONMEM.

```
fetch_etas(lst)
#> # A tibble: 7 x 5
#>   parameter estimate      se         rse shrinkage
#>   <chr>        <dbl>   <dbl>       <dbl>     <dbl>
#> 1 ETA1       0.2      0.0389        19.4        NA
#> 2 ETA2       0.255    0.0463        18.2        NA
#> 3 ETA3       0.1     73          73000          NA
#> 4 ETA4       0.1     73          73000          NA
#> 5 ETA5       0.00001 68      680000000          NA
#> 6 ETA6       0.00001 NA             NA          NA
#> 7 ETA7       0.00001 NA             NA          NA
```

Both RSE and shrinkage rounding can be controlled independently:

```
fetch_etas(lst, digits = 1, shk_digits = 1)
#> # A tibble: 7 x 5
#>   parameter estimate      se         rse shrinkage
#>   <chr>        <dbl>   <dbl>       <dbl>     <dbl>
#> 1 ETA1       0.2      0.0389        19.4        NA
#> 2 ETA2       0.255    0.0463        18.2        NA
#> 3 ETA3       0.1     73          73000          NA
#> 4 ETA4       0.1     73          73000          NA
#> 5 ETA5       0.00001 68      680000000          NA
#> 6 ETA6       0.00001 NA             NA          NA
#> 7 ETA7       0.00001 NA             NA          NA
```

5.3 SIGMA — Residual Error

`fetch_sigmas()` extracts the diagonal of the SIGMA covariance matrix (variance of residual error terms).

```
fetch_sigmas(lst)
#> # A tibble: 2 x 4
#>   parameter estimate      se    rse
#>   <chr>        <dbl>   <dbl>  <dbl>
#> 1 EPS1        0.0769 0.00434   5.64
#> 2 EPS2        0.123  0.199   162.
```

5.4 Objective Function Value

`fetch_ofv()` extracts the OFV from the `#OBJV:` line. For failed or early-terminated runs, it falls back to the workflow footer (`OFV = ...`) if present.

```
fetch_ofv(lst)
#> [1] 8986.318
fetch_ofv(lst, digits = 2)
#> [1] 8986.32
```

5.5 Condition Number

`fetch_condn()` computes the condition number of the correlation matrix of parameter estimates as the ratio of the largest to the smallest eigenvalue. Condition numbers above 1000 suggest numerical instability in the estimation.

```
fetch_condn(lst)
#> [1] 23.30882
fetch_condn(lst, digits = 1)
#> [1] 23.3
```

5.6 All Parameters at Once

`fetch_all()` is a convenience wrapper that calls every parser and returns a named list. Individual failures are caught and returned as `NULL` with a warning, so one problematic section does not abort the entire call.

```
result <- fetch_all(lst)
result$thetas
#> # A tibble: 12 x 4
#>    parameter     estimate       se     rse
#>    <chr>            <dbl>    <dbl>   <dbl>
#>  1 TH_1          34.1     3.37e+ 0 9.88e 0
#>  2 TH_2      387000       5.41e+ 7 1.40e 4
#>  3 TH_3           8.05    5.2 e- 1 6.46e 0
#>  4 TH_4           1.78    1.49e- 1 8.37e 0
#>  5 TH_5           0.498   2.78e- 2 5.58e 0
#>  6 TH_6           0.362   4.81e- 1 1.33e 2
#>  7 TH_7          -0.171   1.35e- 1 7.89e 1
#>  8 TH_8          -0.0314  2.69e- 1 8.57e 2
#>  9 TH_9          -0.163   2.78e- 1 1.71e 2
#> 10 TH10          -0.23    1.88e- 1 8.17e 1
#> 11 TH11           0.00539 1.98e- 2 3.67e 2
#> 12 TH12           0.2     1.90e+74 9.50e76
result$ofv
#> [1] 8986.318
result$condn
#> [1] 23.30882
```

6 Handling Runs Without a Covariance Step

When NONMEM did not complete the covariance step, SE, RSE, shrinkage, and condition number are returned as `NA`. The parameter estimates and OFV are still available.

```
path2 <- system.file("testdata", "theta_no_cov.lst", package = "lstparseR")
lst2  <- read_lst_file(path2)
summary(lst2)
#> <lst> NONMEM listing file
#>   Lines             : 1246
#>   Estimation method: FIRST ORDER CONDITIONAL ESTIMATION WITH INTERACTION
#>   Covariance step  : TRUE
```

```
fetch_thetas(lst2)
#> # A tibble: 12 x 4
#>    parameter     estimate       se     rse
#>    <chr>            <dbl>    <dbl>   <dbl>
#>  1 TH_1          34.1     3.37e+ 0 9.88e 0
#>  2 TH_2      387000       5.41e+ 7 1.40e 4
#>  3 TH_3           8.05    5.2 e- 1 6.46e 0
#>  4 TH_4           1.78    1.49e- 1 8.37e 0
#>  5 TH_5           0.498   2.78e- 2 5.58e 0
#>  6 TH_6           0.362   4.81e- 1 1.33e 2
#>  7 TH_7          -0.171   1.35e- 1 7.89e 1
#>  8 TH_8          -0.0314  2.69e- 1 8.57e 2
#>  9 TH_9          -0.163   2.78e- 1 1.71e 2
#> 10 TH10          -0.23    1.88e- 1 8.17e 1
#> 11 TH11           0.00539 1.98e- 2 3.67e 2
#> 12 TH12           0.2     1.90e+74 9.50e76
```

```
fetch_condn(lst2)
#> [1] 23.30882
```

7 Batch Processing

A common pattern is to parse many `.lst` files from a model development workflow:

```
library(purrr)

files <- list.files("models/", pattern = "\\.lst$", full.names = TRUE)

results <- map(files, \(f) {
  lst <- read_lst_file(f)
  fetch_all(lst)
})

# Combine all THETA tables with run identifier
all_thetas <- map_dfr(results, "thetas", .id = "run")

# Extract OFV for model comparison
ofvs <- map_dbl(results, "ofv")
```

Since `fetch_all()` uses `purrr::safely()` internally, failed runs produce `NULL` elements rather than stopping the pipeline.

8 Function Reference

Function	Description
`read_lst_file()`	Read a `.lst` file into an `lst` object
`fetch_thetas()`	THETA estimates with SE and RSE
`fetch_etas()`	OMEGA diagonal with SE, RSE, and ETA shrinkage
`fetch_sigmas()`	SIGMA diagonal with SE and RSE
`fetch_ofv()`	Objective function value (with footer fallback)
`fetch_condn()`	Condition number from eigenvalues
`fetch_all()`	Run all parsers; return named list
`run_app()`	Launch interactive Shiny application

9 Supported Estimation Methods

lstparseR automatically detects the estimation method from the `.lst` file header. The following methods are supported:

- First Order (FO)
- First Order Conditional Estimation (FOCE)
- First Order Conditional Estimation with Interaction (FOCE-I)
- Stochastic Approximation Expectation-Maximization (SAEM)
- Importance Sampling (IMP)
- Importance Sampling Assisted by Mode A Posteriori (IMPMAP)
- Markov Chain Monte Carlo Bayesian Analysis

10 Interactive Shiny Application

For interactive exploration, lstparseR includes a Shiny application that can be launched with:

```
lstparseR::run_app()
```

The application provides:

- Multi-file upload with instant parsing
- Overview dashboard with parse status summary
- Sortable tables for THETA, OMEGA, and SIGMA parameters
- OFV and condition number summary
- Download buttons for CSV and RDS export

11 Session Info

```
sessionInfo()
#> R version 4.5.2 (2025-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#>   LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=English_Germany.utf8  LC_CTYPE=English_Germany.utf8
#> [3] LC_MONETARY=English_Germany.utf8 LC_NUMERIC=C
#> [5] LC_TIME=English_Germany.utf8
#>
#> time zone: Europe/Berlin
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] lstparseR_0.1.0 testthat_3.3.2
#>
#> loaded via a namespace (and not attached):
#>  [1] compiler_4.5.2    brio_1.1.5        stringr_1.6.0     yaml_2.3.12
#>  [5] fastmap_1.2.0     R6_2.6.1          knitr_1.51        backports_1.5.0
#>  [9] checkmate_2.3.4   tibble_3.3.1      desc_1.4.3        rprojroot_2.1.1
#> [13] pillar_1.11.1     rlang_1.1.7       utf8_1.2.6        cachem_1.1.0
#> [17] stringi_1.8.7     xfun_0.57         fs_2.0.1          pkgload_1.5.0
#> [21] otel_0.2.0        memoise_2.0.1     cli_3.6.5         withr_3.0.2
#> [25] magrittr_2.0.4    digest_0.6.39     rstudioapi_0.18.0 devtools_2.5.0
#> [29] lifecycle_1.0.5   vctrs_0.7.1       evaluate_1.0.5    glue_1.8.0
#> [33] sessioninfo_1.2.3 pkgbuild_1.4.8    rmarkdown_2.31    purrr_1.2.1
#> [37] tools_4.5.2       usethis_3.2.1     pkgconfig_2.0.3   ellipsis_0.3.2
#> [41] htmltools_0.5.9
```